

1. Introducción

En esta actividad trabajamos con la comunicación inalámbrica por medio de Bluetooth en la banda de 2.4 GHz, con el fin de poder enviar y recibir información entre una computadora y un Arduino sin tener que utilizar una conexión por cable. Para realizar esto se utiliza un módulo Bluetooth, como el HC-05 o HC-06, el cual se conecta al Arduino mediante dos pines digitales, haciendo uso de la librería SoftwareSerial, estableciendo así un canal de comunicación serial inalámbrico.

La idea principal de la actividad es comprender de forma práctica cómo funciona este tipo de comunicación en modalidad maestro-esclavo (donde la laptop actúa como maestro y el Arduino como esclavo) y cómo se puede utilizar para controlar dispositivos a distancia. En este caso, la computadora envía comandos hacia el Arduino a través del programa CoolTerm para realizar acciones como encender o apagar un LED, y el Arduino le responde de vuelta a la computadora confirmando la acción realizada para comprobar que la comunicación es correcta.

2. Objetivo

El objetivo de esta actividad es aprender a establecer una comunicación inalámbrica serial entre una computadora y una tarjeta Arduino UNO utilizando un módulo Bluetooth HC-05. Se busca realizar las conexiones físicas correctamente (incluyendo la adaptación de voltaje en la recepción), configurar el módulo mediante comandos AT, emparejar los dispositivos y comprobar la transmisión de datos entre los dos dispositivos.

También se busca comprender de forma práctica el funcionamiento de la comunicación serial emulada por software (SoftwareSerial) y emplear los datos recibidos desde la computadora para ejecutar acciones específicas en el Arduino, como encender o apagar un LED indicador y devolver mensajes de confirmación al usuario a través del software CoolTerm. Con esto se pretende dominar una tecnología de comunicación inalámbrica aplicable en diversos proyectos de ingeniería y automatización.

3. Materiales

Componente	Cantidad	Función
Arduino UNO R3	1	Controlar el funcionamiento del sistema y gestionar la comunicación serial.
Módulo Bluetooth HC-05 (o HC-06)	1	Recibir y transmitir señales de radiofrecuencia (2.4 GHz) para la comunicación inalámbrica.
Computadora o laptop con Bluetooth	1	Dispositivo emisor/receptor para enviar comandos al Arduino y desplegar sus respuestas.
Cable USB A-B	1	Conectar el Arduino a la computadora para cargar el código y alimentarlo.

Componente	Cantidad	Función
Protoboard	1	Conectar los diferentes componentes de una manera más sencilla sin necesidad de soldarlos
Cables jumper	6	Conectar el Arduino al protoboard y unir los componentes entre sí
Resistor de 1 kΩ	1	Resistencia superior del divisor de voltaje para atenuar la señal TX del Arduino.
Resistor de 2 kΩ	1	Resistencia inferior del divisor de voltaje conectada a GND para proteger la entrada RXD del módulo.
Software Arduino IDE	1	Entorno para escribir, compilar y cargar el programa al microcontrolador
Software CoolTerm	1	Terminal serial para enviar cadenas de texto e interactuar inalámbricamente con el Arduino

Nota:

Por simplicidad en la implementación del circuito y para agilizar las pruebas, se utilizó el LED integrado (*on-board*) del Arduino UNO asociado al Pin 13.

4. Metodología

Primero se colocó el Arduino UNO R3, conectado a la laptop vía Cable USB, junto con la protoboard para realizar las conexiones. Se utilizó el pin de 5V del Arduino para alimentar el riel positivo de la protoboard y un pin GND para conectar el riel negativo de tierra común.

4.1 Conexión del Módulo Bluetooth HC-05

Se realizaron las conexiones asegurando que las líneas de transmisión y recepción quedaran unidas correctamente (TX a RX y RX a TX):

- **VCC:** Conectado al riel de 5V de la protoboard.
- **GND:** Conectado al riel de tierra (GND).
- **TXD del módulo:** Conectado directamente al Pin 10 del Arduino (configurado como RX por software).
- **RXD del módulo:** Conectado al nodo central del divisor de voltaje (entre la resistencia de 1kΩ procedente del Pin 11 del Arduino y la resistencia de 2kΩ conectada a GND).

Nota importante (Divisor de Voltaje):

Aunque el módulo Bluetooth HC-05 cuenta con un regulador de voltaje interno que le permite alimentarse con 5V en su pin VCC, sus pines de datos trabajan internamente a un nivel lógico de 3.3V. El pin TXD del módulo envía 3.3V al

Arduino, lo cual es interpretado correctamente como un estado alto (HIGH). Sin embargo, el puerto RXD del módulo no cuenta con regulador y puede dañarse si recibe directamente los 5V que entrega el pin TX del Arduino.

Por esta razón, tal como recomiendan Boylestad y Nashelsky (2013), implementamos un divisor de voltaje compuesto por una resistencia de 1kΩ y otra de 2kΩ entre el pin 11 del Arduino y el pin RXD del módulo. El voltaje resultante se calculó cómo:

$$V_{RX} = 5V \times \left(\frac{2000\Omega}{1000\Omega + 2000\Omega} \right) \approx 3.33V$$

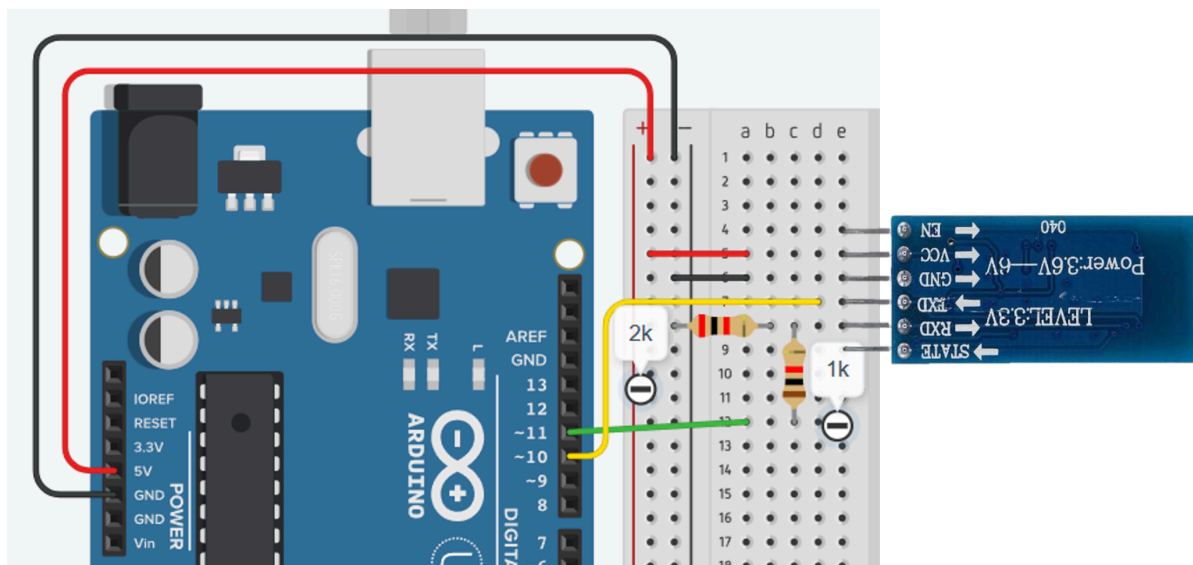
Esto garantiza que el voltaje de la señal entregada al módulo Bluetooth sea segura y no cause problemas.

Componente / Pin	Pin de Arduino	Función del Pin / Esquema de Conexión
VCC (HC-05)	5V	Alimentación principal de 5V para el módulo
GND (HC-05)	GND	Conexión a tierra común del circuito
TXD (HC-05)	Pin 10	Pin 10 (RX por software) -> TXD Módulo
RXD (HC-05)	Pin 11	Pin 11 (TX por software) -> Resistor 1k Ω -> RXD Módulo -> Resistor 2k Ω -> GND
LED	Pin 13	LED integrado de la tarjeta Arduino

4.2 Configuración de Software, Comandos AT y CoolTerm

1. **Configuración inicial en Modo AT:** Antes de conectar, se presionó el botón integrado del HC-05 para entrar en modo de configuración (indicado por un parpadeo lento del LED). Posteriormente, mediante el Monitor Serial de Arduino a 9600 bps se enviaron comandos AT como "AT+NAME=Equipo_5" y "AT+PSWD=1234" para asignar nombre y contraseña al módulo bluetooth y facilitar su acceso. .
2. **Emparejamiento:** Se reconectó el módulo en modo normal (parpadeo rápido), se buscó el dispositivo ("Equipo_5") desde la laptop y se realizó el emparejamiento usando la contraseña asignada previamente.
3. **Identificación de Puerto COM:** En la configuración de Windows (*More Bluetooth Options* → *COM Ports*), se identificó el puerto serie de salida (*Outgoing*).
4. **Conexión en CoolTerm:** Se abrió el programa CoolTerm, se navegó a *Connection* → *Options*, se seleccionó el puerto COM correspondiente a 9600 bps y se presionó el botón *Connect* para iniciar la transmisión inalámbrica.

4.3 Diagrama de conexión



5. Código utilizado

```
1  #include <SoftwareSerial.h>
2
3  SoftwareSerial BTSerial(10, 11); // RX, TX
4
5  void setup() {
6      pinMode(13, OUTPUT);
7      Serial.begin(9600);
8      BTSerial.begin(9600);
9      digitalWrite(13, LOW);
10 }
11
12 void loop() {
13
14     while (BTSerial.available() > 0) {
15
16         char receivedChar = BTSerial.read();
17
18         // Ignorar Enter y salto de línea
19         if (receivedChar == '\r' || receivedChar == '\n') {
20             continue;
21         }
22
23         // Mostrar mensaje recibido en el monitor serial
24         Serial.print("Mensaje recibido: ");
25         Serial.println(receivedChar);
26
27         if (receivedChar == '1') {
28             digitalWrite(13, HIGH);
29             Serial.println("Comando recibido: 1");
30             Serial.println("LED encendido");
31             BTSerial.println("LED encendido");
32         }
33
34         else if (receivedChar == '0') {
35             digitalWrite(13, LOW);
36             Serial.println("Comando recibido: 0");
37             Serial.println("LED apagado");
38             BTSerial.println("LED apagado");
39         }
40
41         else {
42             Serial.println("Comando no valido");
43             BTSerial.println("Comando no valido");
44         }
45     }
46 }
```

6. Explicación del código

El código en Arduino por lo general se divide en las secciones "setup" y "loop". Las instrucciones que sean colocadas en la sección setup se ejecutarán una sola vez cuando el sistema comience, mientras que las instrucciones en la sección loop se repetirán continuamente mientras el Arduino esté encendido.

Para comenzar con el código, primeramente se incluyó la librería `SoftwareSerial.h` mediante la directiva `#include`. Lo que se logra con esta librería es que el Arduino pueda emular comunicación serial en otros pines digitales que no sean los puertos por hardware principales (0 y 1). En este caso se usaron los pines 10 y 11 mediante la instrucción `SoftwareSerial BTSerial(10, 11)`, donde el puerto 10 (RX) recibe información del módulo y el 11 (TX) la transmite.

Después, en la sección setup, primeramente se configuró el puerto 13 como una salida usando `pinMode(13, OUTPUT)`, esto con el fin de controlar el LED indicador integrado en la placa del Arduino. Posteriormente se usó `Serial.begin(9600)`; para fijar la velocidad a la que se comunican el Arduino y la computadora por cable USB a 9600 bps, e igualmente se usó `BTSerial.begin(9600)`; para establecer la velocidad entre el Arduino y el módulo Bluetooth en 9600 bps. Finalmente usamos `digitalWrite(13, LOW)`; para garantizar que cuando el Arduino encienda, el LED comience apagado.

La sección loop se usó principalmente para revisar de manera constante si el módulo Bluetooth recibía información. Para ello, primeramente usamos `while(BTSerial.available() > 0)`, el cual verifica si hay información esperando en la entrada del Bluetooth. En caso de que sí existan datos, la función `BTSerial.read()` se encarga de leer el carácter y guardarlo en la variable `receivedChar`.

Posterior a esto, mediante `if (receivedChar == '\r' || receivedChar == '\n')` el código verifica la presencia de caracteres de salto de línea o retorno de carro para ignorarlos con la instrucción `continue` y evitar lecturas erróneas.

Después, con `Serial.print()` y `Serial.println()` se despliega en el Monitor Serial local del Arduino el mensaje recibido y se analiza el comando:

- Si el comando recibido es '1', se ejecuta `digitalWrite(13, HIGH)`; para encender el LED. Se despliega en el monitor serial local "Comando recibido: 1" y "LED encendido", y simultáneamente mediante `BTSerial.println("LED encendido")`; el Arduino le envía ese mensaje de respuesta a la laptop por vía inalámbrica.
- Si el comando recibido es '0', se ejecuta `digitalWrite(13, LOW)`; para apagar el LED, mostrando "LED apagado" y notificando de vuelta al programa CoolTerm por Bluetooth.
- En caso de que el comando recibido sea cualquier otro carácter, entra en función la condición `else`, la cual despliega "Comando no válido" en ambos monitores seriales, garantizando que el sistema responda únicamente ante las instrucciones válidas '1' y '0'.

7. Resultados

Comando	Mensaje que lee el Arduino	Estado del LED	Respuesta enviada por Bluetooth
1	Comando recibido:1	Se enciende	"LED encendido"
0	Comando recibido:0	Se apaga	"LED apagado"
2	Comando no válido	Se mantiene igual	"Comando no válido"

8. Conclusión

En esta actividad logramos construir, programar y probar con éxito un sistema de comunicación inalámbrica serial entre una computadora y un Arduino UNO utilizando el módulo Bluetooth HC-05.

A través de esta práctica pudimos comprender la importancia de realizar las adaptaciones de voltaje correspondientes mediante un divisor de tensión resistivo, asegurando la integridad del puerto RXD de 3.3V del módulo frente a las salidas de 5V del Arduino. Asimismo, aprendimos a utilizar la librería `SoftwareSerial` para emular puertos seriales secundarios y configurar parámetros mediante comandos AT. Las pruebas ejecutadas en CoolTerm demostraron que el sistema procesa los comandos correctamente y responde en tiempo real con mensajes de retroalimentación. Con esto cumplimos con el objetivo de la actividad, adquiriendo conocimientos prácticos sobre transmisión de datos inalámbrica fundamentales para el desarrollo de proyectos de ingeniería.

9. Referencias

Boylestad, R. L., & Nashelsky, L. (2013). *Electronic devices and circuit theory* (11a ed.). Pearson Education.

Meier, R. (2024). *CoolTerm: Serial port terminal application* (Version 2.1) [Software]. Freeware by Tripos. Recuperado el 4 de octubre de 2026, de <https://freeware.the-meiers.org/>